



1. Single-agent: один агент ведёт задачу целиком

Начнём с самого простого варианта — single-agent. Здесь всю задачу ведёт один агент. Он получает цель, анализирует исходные данные, обращается к доступным инструментам, принимает промежуточные решения и формирует итоговый результат. Для небольшой задачи этого вполне достаточно, потому что нет смысла искусственно разделять работу между несколькими исполнителями.

В Hermes основной агент как раз может выступать таким единым исполнителем. Он способен работать с файлами, терминалом, поиском, памятью и другими подключёнными инструментами. Если задача локальная, например разобраться в причине ошибки в одном модуле, один агент может пройти её полностью без дополнительного распределения ролей. Hermes предоставляет для этого встроенный набор инструментов работы с файлами, терминалом, кодом и другими источниками.

Главное преимущество single-agent режима заключается в простоте. У нас существует один контекст и одна последовательность рассуждений. Агенту не требуется передавать результаты другому агенту и объяснять ему, что уже было сделано.

Но по мере увеличения задачи появляется другая проблема. Один контекст начинает одновременно содержать требования, результаты поиска, изменения кода, результаты тестов и замечания к собственному решению. В определённый момент агенту становится сложнее разделять разные роли внутри одной длинной работы.

2. Multi-agent: задача распределяется между несколькими исполнителями

Multi-agent подход означает, что одна большая задача распределяется между несколькими агентами. Каждый из них получает собственную часть работы и может специализироваться на определённой функции.

Для нас это особенно важно в задачах разработки. Мы можем отделить проектирование решения от непосредственного изменения кода и отдельно провести проверку результата. В таком случае разные этапы не смешиваются внутри одного длинного контекста.

В Hermes такая схема реализуется через механизм делегирования. Основной агент может создавать дочерние экземпляры агентов и передавать им отдельные задачи. Каждый такой субагент получает собственную сессию и может иметь ограниченный набор инструментов. После выполнения дочерний агент возвращает основному агенту итоговый результат своей работы.

То есть multi-agent — это не ситуация, когда несколько моделей просто одновременно отвечают на один вопрос. Здесь появляется распределение ответственности. Один агент может определять, что необходимо сделать, другой выполнять изменение, а третий оценивать результат.

3. Что такое субагент в Hermes

Когда мы говорим о субагенте, я хочу, чтобы вы представляли не дополнительное сообщение внутри того же диалога, а отдельного исполнителя с отдельным рабочим контекстом.

В Hermes субагент создаётся через механизм делегирования. Такой дочерний агент начинает работу с новой историей диалога, получает собственную терминальную сессию и заданный набор доступных инструментов. Промежуточная работа субагента не заполняет основной контекст родительского агента: обратно возвращается итоговое резюме выполнения.

Это достаточно важная особенность. Допустим, субагенту нужно проанализировать большой набор файлов. Он может выполнить множество действий внутри своей собственной сессии, но основной агент не обязан хранить все промежуточные чтения файлов и команды.

Получается своего рода разделение рабочих областей. Основной агент отвечает за общую задачу, а субагент концентрируется на конкретной части работы.

4. Изолированный контекст субагентов

Изоляция создаёт преимущество, но одновременно накладывает важное ограничение. Новый субагент не знает автоматически всего, что ранее обсуждал основной агент.

В документации Hermes это прямо учитывается: дочерний агент начинает с чистого диалогового контекста. Необходимая информация должна быть передана ему вместе с задачей. При работе внутри репозитория Hermes может дополнительно передавать обнаруженный проектный контекст из специальных файлов с инструкциями, но историю разговора родительского агента субагент автоматически не получает.

Поэтому качество мультиагентной системы очень сильно зависит от качества передачи контекста. Если основной агент говорит субагенту просто «исправь проблему», а описание проблемы осталось только в истории основного агента, новый исполнитель может вообще не понимать, что именно требуется.

Это принципиальное отличие от одного длинного диалога. В multi-agent системе границы между исполнителями заставляют нас точнее формулировать ответственность каждого участника.

5. Роль Planner

Теперь разберём роли, которые понадобятся нам дальше. Planner — это агент, отвечающий прежде всего за планирование.

Я бы не рассматривал planner как агента, который обязательно пишет код. Его основная задача состоит в том, чтобы понять исходную цель, определить затрагиваемые компоненты и сформировать последовательность изменений.

Например, если необходимо обновить зависимости проекта, planner должен сначала понять, какая зависимость изменяется, где она используется и какие части программы потенциально потребуют адаптации. После этого появляется план выполнения.

В мультиагентной системе такая роль полезна тем, что отделяет вопрос «что необходимо сделать?» от вопроса «как именно написать изменения?». Это уменьшает вероятность ситуации, когда агент сразу начинает редактировать первый найденный файл, ещё не разобравшись со всей областью задачи.

В Hermes такая логика хорошо сочетается с делегированием, потому что основной агент может использовать результат одного субагента как входные данные для дальнейшей работы других исполнителей.

6. Роль Coder

Coder получает более конкретную ответственность. Его задача уже связана непосредственно с реализацией изменений.

Если planner определил, какие файлы нужно изменить и какие требования необходимо сохранить, coder работает внутри этих границ. Он анализирует соответствующие участки программы, редактирует код и использует доступные ему инструменты разработки.

Для coder особенно важен набор разрешённых инструментов. В Hermes субагенту можно передать ограниченный toolset. Это означает, что исполнитель может получить доступ, например, к терминалу и файлам, но не обязательно ко всем возможностям среды.

Это уже не просто вопрос удобства. Такое ограничение определяет область ответственности агента. Если coder должен заниматься изменением локального проекта, нет необходимости давать ему возможности, которые не относятся к этой задаче.

Поэтому роль определяется не только текстовой инструкцией. Она может дополнительно определяться тем, какие действия технически разрешены конкретному субагенту.

7. Роль Reviewer

Reviewer появляется после выполнения основной реализации и рассматривает результат с другой позиции.

Если coder создавал изменение, его естественная цель заключалась в том, чтобы заставить решение работать. Reviewer, наоборот, должен искать недостатки уже готового результата. Он проверяет соответствие требованиям, возможные побочные эффекты, качество изменения и потенциальные проблемы.

Разделение coder и reviewer полезно ещё и из-за изоляции контекста. Новый субагент может получить результат без всей истории того, почему предыдущий исполнитель принимал определённые решения. В некотором смысле это позволяет провести более независимую проверку.

В Hermes существует даже отдельный подход к разработке через субагентов, где свежие исполнители используются для реализации и последующего многоэтапного ревью. В документации подчёркивается ценность нового контекста для отдельных задач и независимой проверки результата.

Поэтому reviewer в нашей будущей цепочке не является просто последним агентом, который подтверждает работу coder. Его роль заключается именно в критической проверке.

8. Синхронный режим работы

Теперь разберём два режима выполнения. При синхронной организации следующий этап зависит от результата предыдущего.

Для цепочки planner, coder и reviewer это означает, что сначала должен появиться план. Только после этого coder получает сформулированную область изменений. Затем reviewer должен дождаться уже изменённого состояния проекта, чтобы иметь что проверять.

Такой режим нужен тогда, когда между этапами существует зависимость по данным. Reviewer не может полноценно оценить код, который ещё не создан, а coder не может точно следовать плану, которого пока нет.

Синхронный режим проще контролировать, потому что состояние задачи развивается последовательно. Мы всегда можем определить, какой этап уже закончен и какой результат становится входом для следующего.

Но последовательность означает и увеличение общего времени. Если три операции занимают по несколько минут, мы не можем автоматически сложить их параллельно, когда второй этап зависит от первого.

9. Асинхронный и параллельный режим

Асинхронность становится полезной тогда, когда части задачи не требуют результатов друг друга.

Представим большой проект, в котором необходимо отдельно исследовать backend, frontend и конфигурацию зависимостей. Если эти исследования независимы, нет необходимости ждать окончания одного анализа перед началом другого.

Hermes поддерживает запуск нескольких субагентов параллельно через делегирование. В текущей конфигурации по умолчанию допускается до трёх одновременно работающих дочерних агентов, причём значение можно изменять. Каждый из них работает в собственном изолированном контексте.

Но здесь важно не путать параллельность с эффективностью. Если два агента одновременно изменяют один и тот же участок проекта, мы можем получить конфликтующие результаты.

Поэтому асинхронный режим особенно хорошо подходит для независимых исследований и анализа, а последовательный режим — для этапов, где результат одного исполнителя становится основой работы другого.

10. Как будем понимать цепочку Planner, Coder и Reviewer

В рамках дальнейшей работы с Hermes мы будем рассматривать multi-agent систему не как большое количество агентов ради самого количества, а как управляемое разделение ответственности.

Planner будет отвечать за понимание области изменений и подготовку плана. Coder будет работать с реализацией. Reviewer будет отдельно оценивать получившийся результат. Основной агент при этом сможет выполнять роль координатора, который передаёт необходимые данные между этапами и определяет, когда можно переходить дальше.

Hermes хорошо подходит для такого подхода именно потому, что механизм `delegate_task` создаёт отдельных дочерних агентов с изолированными контекстами, собственными рабочими сессиями и настраиваемыми наборами инструментов. Сам Hermes при этом различает делегирование задач, требующих самостоятельного рассуждения, и обычное программное выполнение механических последовательностей через отдельный механизм исполнения кода.

Поэтому главный смысл этой темы не в том, что три агента обязательно лучше одного. Мы используем несколько агентов тогда, когда разделение задачи делает процесс понятнее, позволяет изолировать контекст и создаёт независимую проверку. Если задача короткая и локальная, single-agent остаётся более простым решением. Если же работа естественно разделяется на планирование, реализацию и контроль качества, multi-agent архитектура становится значительно более оправданной.
